



Instituto Superior de Engenharia de Lisboa

Computer Vision and Mixed Reality

Master in Informatics and Multimedia Engineering

2nd Project – Marker Based Augmented Reality

June 2024

Sofia Condesso — 50308

Lectured by
Professor Engineer Pedro Jorge

Contents

1	Introduction	3
1.1	Code organization	3
1.2	Technical Requirements	3
2	a. Camera Calibration	4
2.1	Chessboard Calibration	4
2.1.1	Results	5
2.2	ArUco Board Calibration	6
2.2.1	Results	7
3	b. Detection and camera pose estimation with ArUco library	8
3.1	Results	9
4	c. Registration of virtual objects	9
4.1	Results	9
5	Conclusion	12
	References	12

1 Introduction

This project aims to implement marker based augmented reality and demonstrate the inclusion of virtual elements, aligned with real fiducial markers. To achieve that goal, OpenCV and ArUco marker based library will be used, and this project is divided in three main sections:

- a. **Camera Calibration** - In this step, the intrinsic parameters and distortion coefficients of the camera are computed and saved.
- b. **Detection and camera pose estimation with ArUco library** - Using the intrinsic parameters from the previous step, computes the rotation and translation vectors (extrinsic parameters), given an image.
- c. **Registration of virtual objects** - Using all the parameters, virtual objects are included, according to the detected markers ids.

1.1 Code organization

marker_creator.py - Allows the creation of a board of ArUco markers, of a given dimension.[1]

calib_cam_chessboard.py - Calibration using a chessboard, showed to the webcam. The intrinsic parameters of the camera are saved. It's also possible to undistort an image with a chessboard.

calib_ArUco_boards.py - Calibration using a board of ArUco markers, given a live webcam showing the board. Camera pose is also calculated and printed.

virtual_objects.py - Demonstrates the projection of numpy 3D objects, onto the ArUco board.

1.2 Technical Requirements

In this project, we use the most recent versions of each of this libraries.

```
pip install numpy
pip install matplotlib
pip install opencv-python
pip install opencv-contrib-python [citar]
pip install streamlit
```

Attention: There are some important changes in the latest version, compared to the bibliography provided.[2]

2 a. Camera Calibration

Camera calibration is a crucial step in computer vision tasks, especially when dealing with AR/VR tasks, where the interaction with virtual objects it's common, and therefore, there's a need of ensure that the 3D points in an image correspond accurately to their real-world position.

By calibrating a camera, we obtain the intrinsic parameters (camera matrix and distortion coefficients), which are used later for undistorting images which leads to an improvement the accuracy of image-based measurements.

The focal length (f_x, f_y) and optical centers (c_x, c_y) are the intrinsic parameters, specific to the camera. The focal length and optical centers are used to create a camera matrix (C matrix), which is then used to remove distortion due to the lenses of a specific camera. The camera matrix (3x3) is unique to a specific camera, so once calculated, it can be reused on other images taken by the same camera. [3]

The two main types of distortion are radial distortion, when straight lines appear curved, and tangential distortion, when the image taking lense is not aligned perfectly parallel, in these cases, the image may look nearer than it really is. Other types of distortion are Barrel, Pincushion and Mustache.[4]

2.1 Chessboard Calibration

A chessboard pattern consists of black and white squares arranged in a grid-like structure. The high contrast between the black and white squares makes it an ideal target for automatic corner detection algorithms, allowing us to accurately locate the corners of the chessboard in images.

Inside the folder run the following command:

```
python calib_cam_chessboard.py
```

When running this script, it checks if it is being run as a standalone script (not imported) and initializes the camera and captures real-time images. Now, we can calibrate the camera using a chessboard pattern, by showing a chessboard to the camera for some time, while the system collects sufficient number of images, you can check the progress in the terminal. After collecting sufficient images, it uses the function `calibrate_camera()` to compute the camera matrix (C) and distortion coefficients. With this information we can start to stream undistorted video, estimate poses for a given frame and perform projections of virtual objects.

The following parameters are needed:

- 1) The batch of frames to use in calibration (frames).
- 2) Height (h) and width (w).
- 3) Termination criteria for the corner finding algorithm (criteria).

The function[3] performs the following tasks:

1. Converts the frame from BGR to grayscale.
2. Finds chessboard corners using `cv.findChessboardCorners()`.
3. Refines corner locations using `cv.cornerSubPix()`.
4. Calibrates the camera using `cv.calibrateCamera()`.
5. Saves the results and prints the camera matrix, distortion coefficients to a file named 'calib.npz'. The obtained data can be used for undistorting images captured by the camera.
6. Undistorts the frame using `cv.undistort()`.

2.1.1 Results

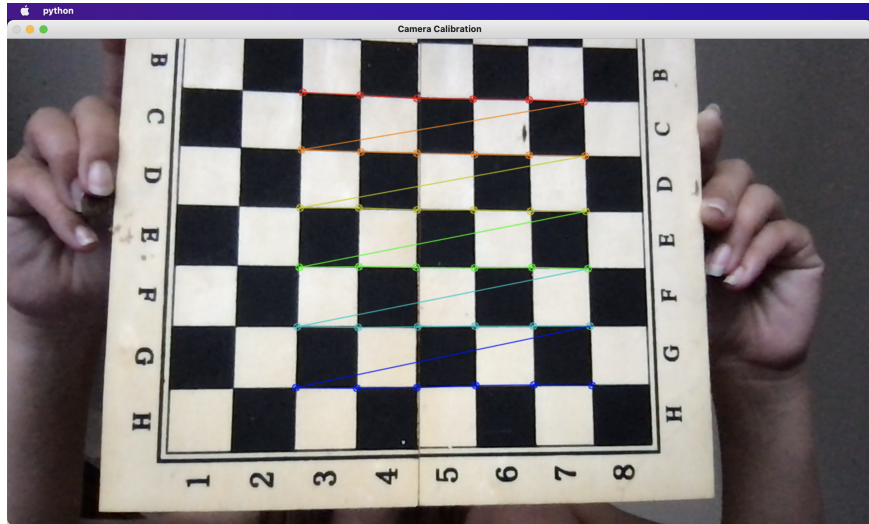


Figure 1: Frame with identified chessboard corners, during the calibration phase, considering chessboard with dimension (6,6) and winSize (40, 40).

For these experiments, 100 pictures were collected through webcam, while the user varies the chessboard position. As we can observe on the following table, the intrinsic parameters have high variance, but the measured values seem to follow a proportional pattern, with f_x and f_y very close to each other, and with c_y being a little more than an half of c_x , except for the fourth experiment, where c_y is less than an half of c_x . This may be explained by the winSize not being big enough (it's the smallest winSize experimented).

Board Dimension (h,w)	cornerSubPix winSize	fx	fy	cx	cy
(5, 4)	(11, 11)	996.67	981.38	593.08	334.79
(5, 4)	Not used	1133.79	1140.42	632.61	372.19
(5, 4)	(10, 8)	989.61	1001.91	611.13	351.12
(3, 3)	(6, 6)	103.86	133.00	898.86	217.66
(3, 3)	(40, 40)	2490.40	2439.91	526.38	370.00
(3, 3)	(20, 20)	989.24	1001.58	633.90	332.96
(6, 6)	(40, 40)	993.16	1004.99	670.98	334.52

Table 1: Results of intrinsic parameters, for some experiments varying the board dimension and the winSize parameter from cv2.cornerSubPix method.

It wasn't possible to calibrate the camera with the real dimensions of the chessboard (8x8), because it wasn't recognizing the pattern, even after trying several positions and distances from the camera. The biggest size of chessboard dimensions, that was possible to achieve calibration, during these experiments, was the size of 6x6.

2.2 ArUco Board Calibration

An ArUco marker is a synthetic square marker composed by a wide black border and an inner binary matrix which determines its identifier. The ArUco fiducial marker can also be used to calibrate a camera. A fiducial marker is a landmark added to a scene to facilitate locating points.

Calibrating using ArUco is much more versatile than using traditional chessboard patterns, since we don't need to include the complete board view, and we have each marker well identified, one by one.[4]

To calibrate the camera through this approach, run the following command:

```
python calib.ArUco.boards.py
```

And, after some time showing and moving the pattern, press "q" key to quit, the program will use the function `calibrate_aruco()`, that uses OpenCV library with AruCo module for calibrating a camera using an AruCo marker grid. The following parameters can be changed:

- 1) The batch of frames to use in calibration (frames).
- 2) Height (h) and width (w).
- 3) Floats for marker_length and marker_separator.
- 4) A dictionary of aruco markers.

The code then proceeds as follows:

1. Initializes AruCo dictionary, detector parameters, and creates an AruCo GridBoard object.

2. Initializes empty lists for storing counter (number of detected markers), corners (2D image points), ids (marker IDs), and final images.
3. Uses a loop to iterate through each image, detects the markers using 'aruco.detectMarkers' function, and stores the corners and ids in their respective lists. Also, adds the image to the final_images list.
4. Once all images have been processed, apply camera calibration using 'aruco.calibrateCameraAruco' function.
5. Save the camera parameters. We can then estimate pose (position and orientation) of each marker using 'aruco.estimatePoseSingleMarkers' function.

2.2.1 Results

As an observation, from an error I got while experimenting, the value of markerSeparation cannot be bigger than 70% of markerLength.

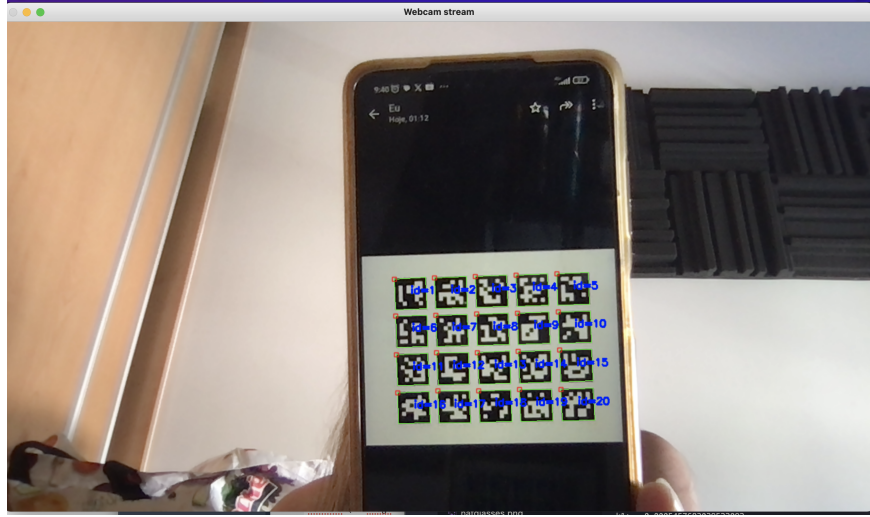


Figure 2: Markers identification, during the calibration phase, using a board of ArUco markers 5x4.

For the following experiments, 100 pictures were collected through webcam, while the user varies the ArUco board position. To get the markerLength and markerSeparation parameters, we must measure one side of a marker (markerLength) and the white space between markers, in centimeters.

The experiments marked with a '*' correspond to the ones that have the real values for markerLength and markerSeparation. The other experiments were to

test a pair of values that would meet the other results in the middle, in terms of the values for those parameters.

Board Dim. (h,w)	ArUco Dictionary	Marker (Length, Separation)	fx	fy	cx	cy
(2, 2)*	6X6_250	(1.8, 0.5)	1414.93	565.34	274.31	727.06
(5, 3)*	7X7_50	(0.7, 0.3)	607.74	726.87	389.93	651.90
(5, 4)*	6X6_250	(0.8, 0.3)	339.08	119.20	464.32	417.66
(2, 2)	6X6_250	(1.2, 0.5)	246.37	41.91	480.74	490.44
(5, 3)	7X7_50	(1.2, 0.5)	450.04	102.35	95.33	452.32
(5, 4)	6X6_250	(1.2, 0.5)	461.00	108.67	513.69	411.05

Table 2: Results of intrinsic parameters, for experiments varying the board dimension, the markerLength and the markerSeparation parameters of GridBoard.

Observing the results, fy tends to be, approximately, one third of the value of fx, except in the second experiment. But this is not sufficient for conclusions about the best approaches and parameters.

On the overall, though more complex, using ArUco marker boards allows the achievement of multiple use cases, where high fidelity with real-world coordinates is imperative, due to its flexibility and tracking of each marker.

In the next section, some of these calibrations will be tested with virtual 3D objects.

3 b. Detection and camera pose estimation with ArUco library

As a result of the calibration, we got the camera matrix, a 3x3 matrix with the focal distances (fx, fy) and camera center coordinates (cx, cy), and we also got the distortion coefficients, a vector that models the distortion produced by the camera.

After detecting the markers, we're going to use them to estimate the camera pose, so we can project 3D objects with the right rotation and translation, according to the intrinsic parameters achieved in the previous section.

The camera pose relative to the marker is a 3D transformation from the marker coordinate system to the camera coordinate system, as we going to see in the next section. This function. When estimating the pose with ArUco markers, the pose of each marker was estimated, individually, using 'estimatePoseSingleMarkers()' function, which uses the corners, the markerLength, the camera matrix and distortion coefficients, to compute the rotation and translation vectors.[5]

3.1 Results

Observing the results of intrinsic parameters calculated while calibrating the camera, we can conclude that, for each type of calibration, we need to do a more extensive research for the best parameters needed by the functions used during the process. That can be achieved by using more images for calibration and by moving the pattern in different directions, to have more information. Once that study is done for a specific use case, we should achieve better pose estimations, and therefore, perform better on the task from the next section.

4 c. Registration of virtual objects

The script `virtual_objects.py` performs ArUco marker real-time detection and rendering of 3D objects. Using ArUco boards instead of single markers allows some markers to be occluded. Some of the functions implemented to achieve this goal are:

Main Function: Starts by loading previously calculated calibration parameters (mtx and dist) using `'load_calibration_parameters()'`. Then it calls the `'detect_and_register_virtual_objects()'` function to start the real-time ArUco marker detection and rendering process.

Detect and Register Virtual Objects: The `'detect_and_register_virtual_objects'` function captures video from the camera, detects markers in real-time using `aruco.detectMarkers()`, estimates their pose (position and orientation) using `aruco.estimatePoseSingleMarkers()`, and renders the virtual objects at the correct location using the `'draw_virtual_objects()'` function.

Draw Virtual Objects: The `'draw_virtual_objects()'` function takes the input frame, registration vectors (rvecs and tvecs), intrinsic and distortion parameters (mtx and dist), marker IDs, and draws 3D virtual objects on the given image based on the provided information.

4.1 Results

In these examples, we can observe the perspective projection of the cubes, in ArUco boards with different dimensions, and also using different ArUco dictionaries.

Keeping parameter for the size of the cube for both experiments:

- In the 2x2 board experiment (Figure 3) we can observe that the cubes turn into parallelepiped shape, when moving the marker board, this can be due to poorer calibration, since it is a small board.
- With the 5x3 board, the cubes tend to keep a shape that is more loyal to the real-world.

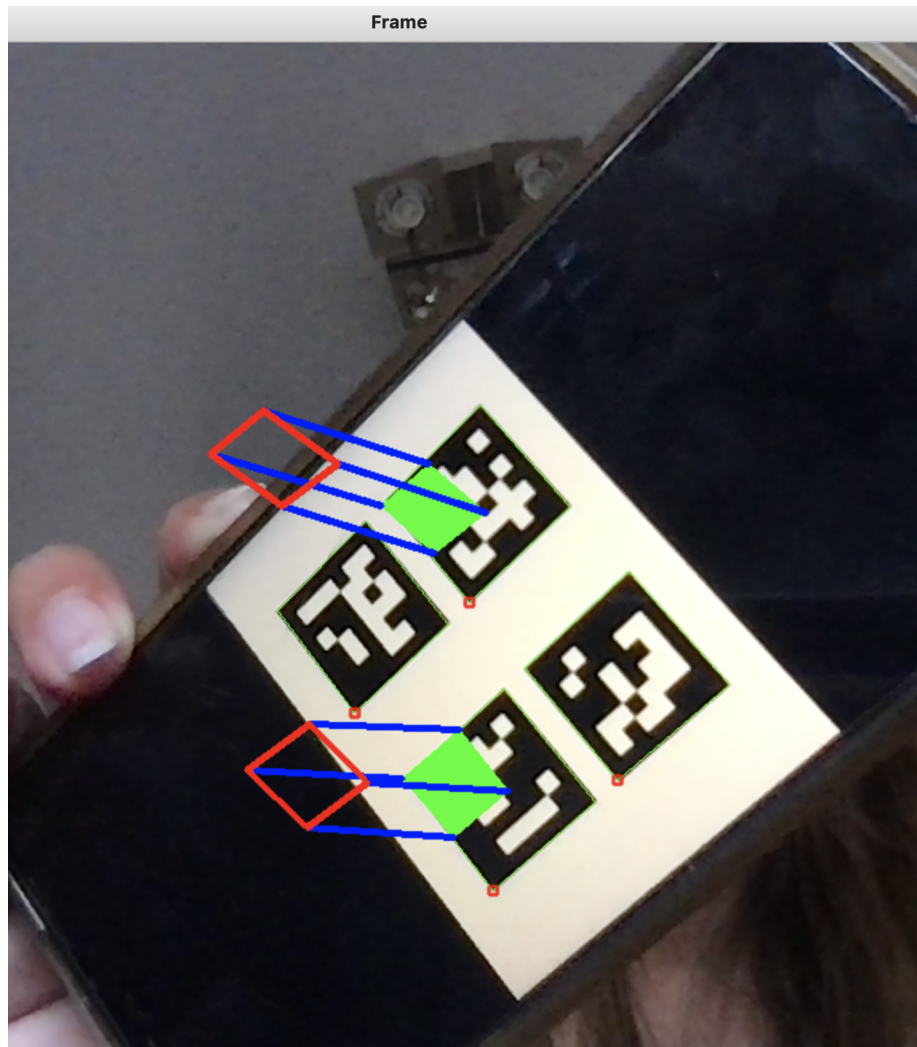


Figure 3: Registration of cubes in a 2x2 board of 6x6_250 ArUco markers with ids 1 and 4.



Figure 4: Registration of cubes in a 5x3 board of 7x7_50 markers with ids 1, 4, 7 and 9.

Multiple boards and settings were tested, the tool performs seemingly well on most of them. Sometimes the stream stops due to an exception, that causes some heavy computation, that arises from the process, associated, most of the times, with too much approximation from the board to the camera.

5 Conclusion

In this project, augmented reality tasks were explored, developed and tested, using ArUco integrated in OpenCV Python library. This work can surely be helpful in further projects.

The versions of libraries change a little in syntax, making the code not executable with other versions that aren't the latest. The documentation for Python is not very extensive, so finding the correct syntax was one of the major challenges.

Other challenge I faced, was to get proper 3D objects to do register of all distinct objects in the respective marker, so I decided to only demonstrate how to associate an object.

Further work can be done in this project, like:

- Build a UI to perform these kind of tasks and experiments.
- Expand the code to include more complex virtual objects or animations for different markers.
- Improve code documentation.
- Test the code on different setups such as camera resolutions and lighting conditions to evaluate its performance and robustness.
- Compare distorted and undistorted images.

This project can also serve as a baseline for many different AR apps. One idea could be to create an immersive AR chess experience. In conclusion, accurate camera calibration is vital for immersive VR experiences and OpenCV library provides efficient tools for camera calibration in Python.

References

- [1] <https://www.makeuseof.com/python-aruco-marker-generator-how-create/>
- [2] <https://medium.com/@ed.twomey1/using-charuco-boards-in-opencv-237d8bc9e40d>
- [3] https://docs.opencv.org/master/dc/dbb/tutorial_py_calibration.html
- [4] <https://medium.com/vacatronics/3-ways-to-calibrate-your-camera-using-opencv-and-python39>
- [5] <https://automaticaddison.com/how-to-perform-pose-estimation-using-an-aruco-marker/>
- [6] <https://www.uco.es/investiga/grupos/ava/node/26>
- [7] https://docs.opencv.org/4.9.0/d9/d6d/tutorial_table_of_content_aruco.html

- [8] <https://chev.me/arucogen/>
- [9] https://mecaruco2.readthedocs.io/en/latest/notebooks_rst/Aruco/aruco_basics.html
- [10] https://docs.opencv.org/4.9.0/db/da9/tutorial_aruco_board_detection.html
- [11] https://docs.opencv.org/4.9.0/d5/dae/tutorial_aruco_detection.html
- [12] https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html